

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Системное программирование (СП)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

ПРОГРАММНОЕ СРЕДСТВО
«Файловый менеджер с возможностью просмотра файлов на
устройствах в одной локальной сети»

БГУИР КП 1-40 01 01 101 ПЗ

Студент:

Бетень К.С.

Руководитель:

Деменковец Д.В.

Минск 2025

Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Факультет компьютерных систем и сетей

УТВЕРЖДАЮ
Заведующий кафедрой

(подпись)

20__ г.

ЗАДАНИЕ
по курсовому проектированию

Студенту Бетене Константину Сергеевичу

1. Тема проекта Программное средство "CRM-system"
2. Срок сдачи студентом законченного проекта 06 июня 2025 г.
3. Исходные данные к проекту редактор исходного кода VS CODE, система контроля версий git, платформа для хостинга GitHub, платформа Node.js и язык программирования Java Script.
4. Содержание расчетно-пояснительной записки (перечень вопросов, которые подлежат разработке) _____
Введение
1. Анализ предметной области
2. Проектирование программного средства
3. Разработка программного средства
4. Тестирование программного средства
5. Руководство пользователя
6. Список использованных источников
5. Перечень графического материала (с точным обозначением обязательных чертежей и графиков)
Чертеж А1 по ГОСТ 19.701-90

6. Консультант по проекту (с обозначением разделов проекта) А.Ю. Шамына

7. Дата выдачи задания 24 февраля 2025 г.

8. Календарный график работы над проектом на весь период проектирования (с обозначением сроков выполнения и трудоемкости отдельных этапов):

разделы 1,2 к 01.03 – 15 %;

раздел 3 к 21.03 – 10 %;

разделы 4,5 к 03.04 – 10 %;

разделы 6,7 к 15.04 – 35 %;

раздел 8 к 30.04 – 5 %;

раздел 9 к 03.05 – 10 %;

оформление пояснительной записки и графического материала к 13.05 – 15 %

Защита курсового проекта с 01.06 по 06.06

РУКОВОДИТЕЛЬ А.Ю. Шамына
(подпись)

Задание принял к исполнению К.С. Бетенья
(дата и подпись студента)

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Аналогии программного средства	6
1.2 Постановка задачи	8
2 Разработка структурной схемы	9
2.1 Структура программы	9
2.2 Интерфейс программного средства	9
2.3 Проектирование архитектуры программного средства	10
2.4 Проектирование функциональных требования	12
3 Разработка программного средства	15
3.1 Работа программного средства	15
3.2 Модели данных	16
4 Тестирование программного средства	18
5 Руководство пользователя	20
5.1 Интерфейс программного средства	20
5.2 Управление программным средством	22
Заключение	24
Список использованных источников	25
Приложение А (обязательное) Исходный код	26

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и усиления конкуренции на рынке, особое значение приобретает качество взаимодействия компании с клиентами. Успешность коммерческой деятельности всё в большей степени определяется не только качеством предоставляемых товаров и услуг, но и способностью предприятия эффективно выстраивать и поддерживать долгосрочные взаимоотношения с потребителями. В этом контексте важную роль играют системы управления взаимоотношениями с клиентами, или CRM-системы.

CRM-система – это комплекс организационных, технических и программных решений, направленных на сбор, хранение, анализ и использование информации о клиентах и взаимодействии с ними. Такие системы позволяют централизованно управлять данными о клиентах, автоматизировать процессы продаж, маркетинга и обслуживания, а также повышать уровень удовлетворённости клиентов и, как следствие, рентабельность бизнеса. Они способствуют формированию единого информационного пространства, доступного всем участникам бизнес-процессов.

Основной задачей CRM-системы является построение устойчивых и взаимовыгодных отношений между организацией и её клиентами, основанных на понимании потребностей и индивидуальном подходе. Это достигается за счёт анализа истории взаимодействий, мониторинга активности, управления обращениями, планирования контактов и последующего анализа результатов. Таким образом, CRM-системы становятся важным инструментом стратегического управления, позволяющим организациям адаптироваться к меняющимся условиям рынка и повышать свою конкурентоспособность.

Развитие CRM-подхода также оказывает значительное влияние на внутренние процессы в компании. Он способствует улучшению взаимодействия между отделами, повышению прозрачности процессов, ускорению принятия решений и повышению общей эффективности работы персонала. Внедрение CRM-системы требует системного подхода и глубокого анализа бизнес-процессов, что делает её не только технологическим, но и управленческим инструментом.

Таким образом, тема CRM-систем является актуальной и значимой в контексте цифровизации бизнеса и стремления компаний к устойчивому развитию. Изучение теоретических основ и функциональных возможностей CRM-систем позволяет глубже понять механизмы эффективного взаимодействия с клиентами и сформировать представление о современных подходах к управлению в условиях рыночной экономики.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Аналоги программного средства

1.1.1 Сервис для управления бизнесом «Битрикс 24»

Одна из наиболее широко распространенных реализаций будущего программного средства является сервис для управления бизнесом «Битрикс 24».

Многофункциональная платформа для управления бизнесом, включающая CRM, задачи, чаты, телефонию, документы и автоматизацию процессов. Пользователь получает доступ к единому рабочему пространству, где можно вести базу клиентов, управлять сделками, общаться с коллегами и контролировать задачи. Интерфейс ориентирован на обычного пользователя без технических знаний, с возможностью гибко настраивать воронки продаж, автоматизировать действия и интегрироваться с популярными сервисами (почта, мессенджеры, телефония).

Рассмотреть данный сервис можно на рисунке 1.1.

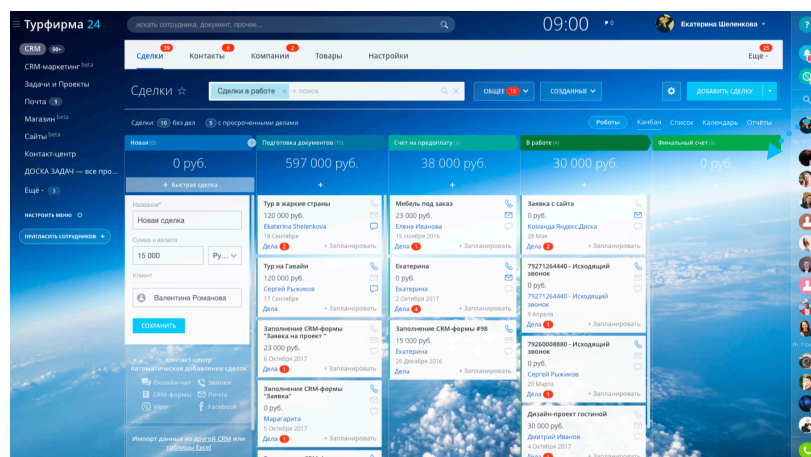


Рисунок 1.1 – Программное средство «Битрикс 24»

К преимуществам системы можно отнести богатый функционал в базовой версии продукта, наличие бесплатного тарифа, облачную версию, доступ с мобильных устройств и широкие возможности автоматизации. Минусы — перегруженность интерфейса, особенно для новых пользователей, ограниченная гибкость на бесплатном тарифе, а также заметное снижение скорости работы при большом объеме данных или использовании коробочной версии без оптимизации.

1.1.2 Программное средство «Zoho CRM»

Ещё одной реализацией является программное средство «Zoho CRM».

«Zoho CRM» — это облачная CRM-система, ориентированная на автоматизацию продаж, маркетинга и клиентского сервиса. Пользователь работает в чистом и понятном интерфейсе, где легко отслеживать

взаимодействие с клиентами, управлять лидами, сделками и задачами. Система предлагает гибкую настройку воронок продаж, автоматические сценарии, аналитику и интеграции с другими продуктами Zoho и внешними сервисами, включая почту, мессенджеры и соцсети.

Интерфейс программного средства представлен на рисунке 1.2.



Рисунок 1.2 – Программное средство «Zoho CRM»

1.1.3 Система взаимоотношений «1C:CRM»

Одним из наиболее популярных и широко используемых программных решений для управления взаимоотношениями с клиентами на рынке является «1C:CRM» — специализированный модуль в составе платформы «1C:Предприятие». Это программное средство ориентировано на автоматизацию процессов взаимодействия с клиентами и партнёрами, управление продажами, маркетингом и сервисным обслуживанием. Оно предназначено для использования как в малом и среднем бизнесе, так и в крупных организациях с развитой системой клиентских коммуникаций.

Рассмотреть данный сервис можно на рисунке 1.3.

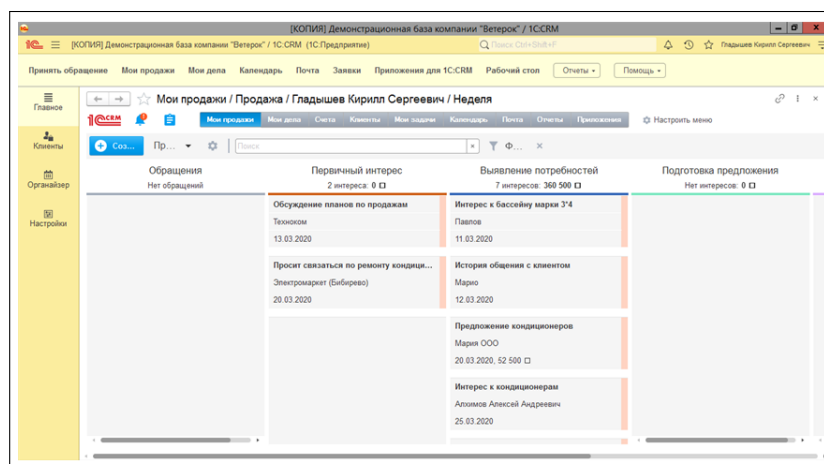


Рисунок 1.3 – Программное средство «1C:CRM»

«1С:CRM» обеспечивает централизованное хранение информации о клиентах, историях взаимодействий, сделках, заявках, контактах и других аспектах делового общения. Система позволяет вести учёт и планирование задач, контролировать исполнение поручений, сегментировать клиентскую базу, анализировать эффективность маркетинговых мероприятий и прогнозировать продажи.

Система гибко настраивается под нужды конкретной компании и интегрируется с другими продуктами линейки 1С, а также с внешними сервисами (например, почтовыми клиентами, IP-телефонией и сайтами).

1.2 Постановка задачи

Цель курсовой работы – разработка своей универсальной CRM системы в виде веб-приложения. На основе проведённого анализа требуется реализовать следующие функции:

- сервис для работы с пользователями;
- сервис для работы с задачами;
- сервис веб-приложения;
- сервис базы данных;
- облачное файловое хранилище;
- сервис логирования;
- механизм межсервисного взаимодействия;

При разработке программного средства будет использована среда разработки Visual Studio Code [1] с использованием языка программирования JavaScript [2] и платформы Node.js [3]. Вспомогательными инструментами будут являться Docker [4] и Nginx [5] для создания контейнеров и проксирования запросов. Для работы с базой данных будет использоваться PostgreSQL [6].

2 РАЗРАБОТКА СТРУКТУРНОЙ СХЕМЫ

2.1 Структура программы

Было принято решение, что при разработке данного программного средства будут использоваться восемь модулей:

- Frontend – модуль веб-приложения с которым будет взаимодействовать пользователь;
- TaskService – модуль для работы с задачами пользователя;
- AuthService – модуль для работы с пользователями и их аутентификацией и авторизацией;
- LogService – модуль для обработки и записи ошибок, которые возникают в процессе работы системы;
- Kafka – модуль для межсервисного взаимодействия;
- Kibana – модуль для визуализации логов;
- PostgreSQL – модуль для базы данных где будет храниться вся информация приложения;
- Nginx – модуль для проксирования запросов к сервисам;

Каждый сервис будет реализован в виде Docker-контейнера с целью упрощения развертывания и настройки системы.

2.2 Интерфейс программного средства

Для проектирования интерфейса программного средства было принято решение использовать, как основу, библиотеку компонентов Material UI [7]. Данная библиотека предоставляет готовые компоненты, которые позволяют быстро и удобно создавать интерфейсы, а также поддерживает адаптивный дизайн, что позволяет использовать приложение на различных устройствах.

Для проектирования архитектуры было принято решение использовать паттерн Feature-Sliced Design.

2.2.1 Блочные компоненты

Блочные компоненты представляют собой основные элементы интерфейса, которые используются для построения страниц приложения. В данном программном средстве будут использоваться следующие блочные компоненты:

- Stack - группировка части информации;
- Card - карточка, которая содержит информацию;
- Grid - для работы с сетками;
- Table - вывод информации в виде таблиц;

Блочные компоненты могут содержать в себе другие компоненты, а также стили и логику, которые необходимы для их работы.

2.2.2 Компоненты управления

Компоненты управления представляют собой элементы интерфейса, которые позволяют пользователю взаимодействовать с приложением. В данном программном средстве будут использоваться следующие компоненты управления:

- Button - кнопка, которая выполняет действие;
- Input - поле ввода информации;
- Select - выпадающий список;

Для работы с формами будет использоваться стандартная библиотека react-hook-form, которая позволяет удобно и быстро работать с формами, а для работы с валидацией будет использоваться библиотека Yup, которая позволяет удобно и быстро работать с валидацией форм.

2.2.3 Компоненты отображения

Компоненты отображения представляют собой элементы интерфейса, которые отображают информацию пользователю. В данном программном средстве будут использоваться следующие компоненты отображения:

- Typography - для отображения текста;
- Avatar - для отображения аватаров пользователей;
- Icon - для отображения иконок;
- Snackbar - для отображения уведомлений пользователю;
- CircularProgress - для отображения индикатора загрузки;
- Tooltip - для отображения подсказок;

2.3 Проектирование архитектуры программного средства

При проектировании программного средства изначально была выбрана микросервисная архитектура, которая позволяет разбить приложение на независимые модули, которые могут быть разработаны и протестированы отдельно друг от друга. Это позволяет упростить процесс разработки и тестирования, а также улучшить масштабируемость приложения.

2.3.1 Схема взаимодействия микросервисов

В процессе проектирования было определено, что необходимо реализовать следующие микросервисы:

- микросервис для работы с пользователями;
- микросервис для работы с задачами;
- микросервис для работы с логированием;

Под каждый микросервис будет разрабатываться в отдельной ветке моно-репозитория с настроенным CI/CD для проверки сборки микросервиса и отправки его в Docker Hub хранилище для последующего использования.

Схема взаимодействия микросервисов представлена на рисунке 2.1.

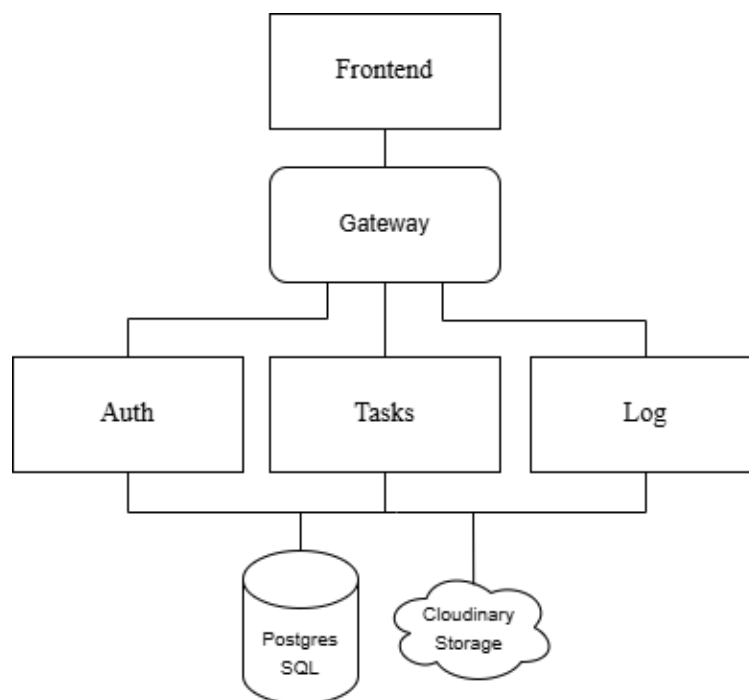


Рисунок 2.1 – Общая схема взаимодействия микросервисов

Для упрощения работы с API на стороне клиент было решено настроить и использовать API Gateway, который будет служить проху сервером для всех запросов к микросервисам.

2.3.2 Брокер сообщений

Для распределения нагрузки между микросервисами и обеспечения их взаимодействия без нагрузки на Gateway был внедрён брокер сообщений Kafka [8]. Для правильной работы с брокером сообщений необходимо также использовать Zookeeper, который будет хранить конфигурацию брокера и обеспечивать его работу.

Схема работы брокера сообщений представлена на рисунке 2.2.

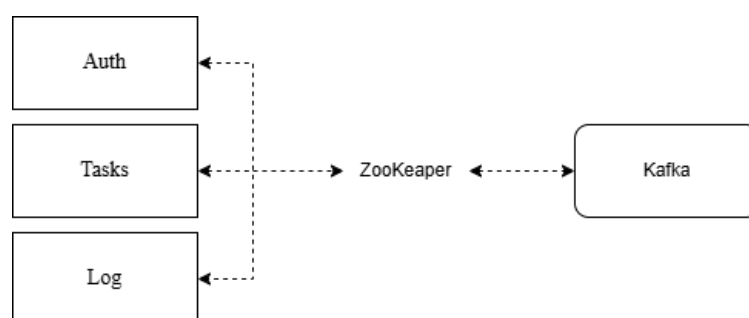


Рисунок 2.2 – Схема взаимодействия брокера сообщений

Zookeeper служит средством распределения нагрузки между брокерами путём выделения лидера который будет обрабатывать все запросы, что является необходимым условием для работы брокера сообщений.

2.3.3 Логирование

Для удобного и быстрого получения информации о состоянии системы в неё внедряются системы логирования. В данном случае будет использоваться ELK набор, состоящий из следующих компонентов:

- Elasticsearch - система хранения и поиска данных;
- Logstash - система сбора и обработки логов;
- Kibana - система визуализации данных;

Схема работы системы логирования представлена на рисунке 2.3.

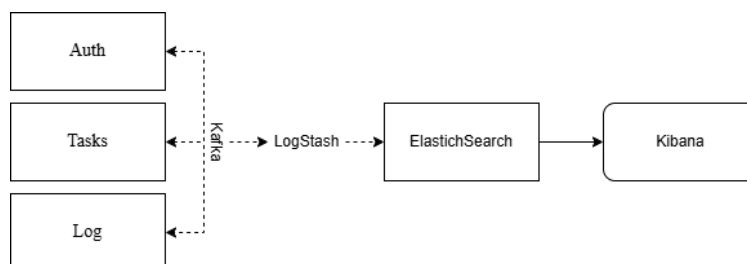


Рисунок 2.3 – Схема работы логирования

Logstash перехватывает сообщения из Kafka с определённым заголовком и отправляет их в Elasticsearch, который хранит их в своей базе данных. Для более удобного поиска, фильтрации и визуализации данных можно использовать Kibana [9]. Функционал Kibana позволяет визуализировать данные в виде графиков, таблиц и прочих элементов, что позволяет быстро и удобно получить информацию о состоянии системы.

2.4 Проектирование функциональных требования

Функционал программного средства – ключевая точка разработки программного средства, составляющей которого являются алгоритмы. В связи с этим, программное средство «Файловый менеджер с возможностью просмотра файлов на устройствах в одной локальной сети» должно обладать некоторыми алгоритмами:

- алгоритм создания JWT токена;
- алгоритм сохранения файла в облачное хранилище;
- алгоритм отправки уведомления в систему логирования;

2.4.1 Алгоритм создания JWT токена

JWT токен [10] – это JSON Web Token, который используется для передачи информации между клиентом и сервером в виде JSON объекта. Алгоритм создания JWT токена представлен на рисунке 2.4.

JWT токен состоит из трех частей: заголовка, полезной нагрузки и подписи. Заголовок содержит информацию о типе токена и алгоритме подписи. Полезная нагрузка содержит информацию о пользователе и сроке

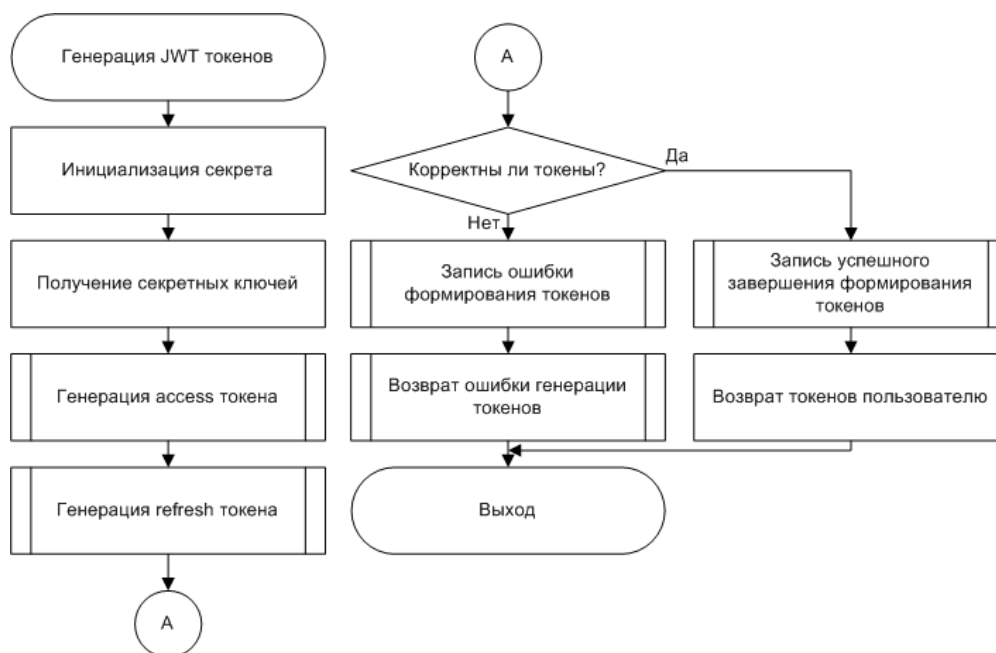


Рисунок 2.4 – Алгоритм создания JWT токена

действия токена. Подпись используется для проверки целостности токена и его подлинности.

Access токен – это токен, который используется для доступа к защищенным ресурсам. Refresh токен – это токен, который используется для получения нового JWT токена, когда срок действия старого токена истек. Refresh токен имеет больший срок действия, чем Access токен.

2.4.2 Алгоритм сохранения файла в облачное хранилище

Облачное хранилище – это сервис, который позволяет хранить данные в интернете и получать к ним доступ из любой точки мира. Облачное хранилище позволяет хранить данные в защищенном виде и обеспечивает доступ к ним только авторизованным пользователям.

Алгоритм сохранения файла в облачное хранилище представлен на рисунке 2.5.

Алгоритм сохранения файла в облачное хранилище начинается с инициализации необходимых компонентов: имя облачного хранилища, API ключ и API секрет. После этого инициализируется объект, через который будет происходить взаимодействие с облачным хранилищем. В объекте настраиваются параметры облачного хранилища: папка и формат файлов. После этого используя внутренние библиотеки языка программирования можно выполнить функцию для загрузки файла в облако. Результат работы это свойства опубликованного файла, которые передаются обработчику запроса для дальнейшего взаимодействия.

2.4.3 Алгоритм отправки уведомления в систему логирования

Система логирования – это система, которая позволяет отслеживать события, происходящие в программном обеспечении. Система логирования

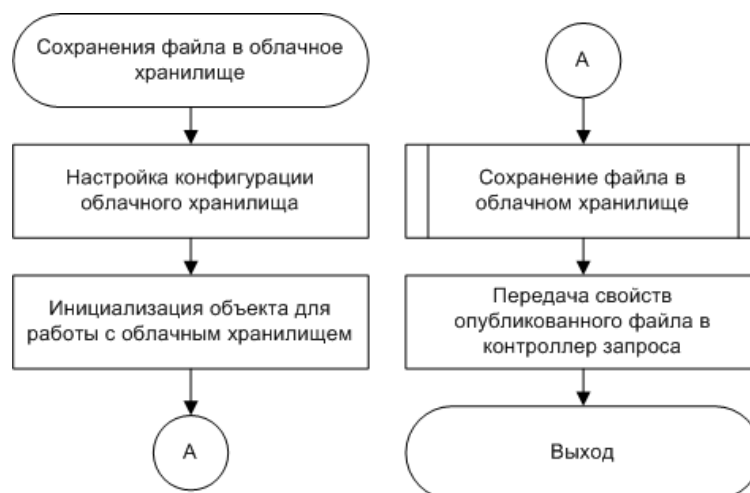


Рисунок 2.5 – Алгоритм сохранения файла в облачное хранилище

позволяет сохранять информацию о событиях, происходящих в программном обеспечении, и предоставляет возможность анализировать эту информацию. Алгоритм отправки уведомления в систему логирования представлен на рисунке 2.6.

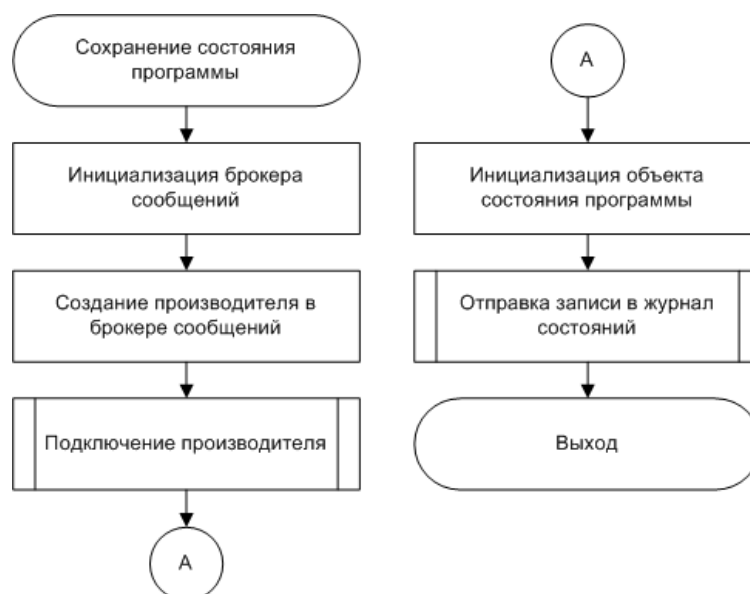


Рисунок 2.6 – Алгоритм отправки уведомления в систему логирования

Перед началом работы необходимо инициализировать брокер сообщений и создать производителя для генерации нового состояния в журнале событий программного средства. Далее, при возникновении нового сообщения запускается производитель и формируется объект, который хранит текущее состояние системы. Остаётся только создать событие в брокере через производителя и передать новое сообщение.

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

Для разработки программного средства были использованы язык программирования JavaScript на платформе Node.js. С целью улучшения поддержки исходного кода в ходе разработки активно использовался комплект библиотек, предлагаемый выбранной языковой платформой.

3.1 Работа программного средства

К основному функционалу программного средства можно отнести следующие компоненты:

- регистрация нового пользователя;
- вход в аккаунт;
- создание новой задачи;

3.1.1 Регистрация нового пользователя

Регистрация нового пользователя начинается с получения переданных в запросе параметров и проверка их на корректность. После получения необходимых параметров, функция проверяет логин пользователя на уникальность: нет ли уже такого логина в системе. Если данный логин свободен, то необходимо получить логотип пользователя и зашифровать пароль. После этого создаётся новый пользователь. После этого формируется запись в журнал состояний, что пользователь успешно создан и возвращается код успешного завершения запроса пользователю.

Если в процессе работы алгоритма произошла непредвиденная ошибка, то будет создана запись в журнал об ошибке и на клиент вернётся код плохого завершения запроса и краткое описание проблемы.

3.1.2 Вход в аккаунт

Если у пользователя уже есть зарегистрированный в системе аккаунт, то ему необходимо в него войти. При запуске приложения у пользователя появляется страница авторизации в систему, где необходимо ввести свой логин и пароль.

После того как пользователь ввёл логин и пароль, формируется запрос к микросервису авторизации на проверку. Если логин и пароль совпадают с существующими в системе, то пользователя пускаю в систему под его данными.

3.1.3 Создание новой задачи

Для создания задачи авторизованный пользователь должен заполнить, как минимум 3 поля:

- название задачи;
- дата начала задачи;
- дата окончания задачи;

Существуют необязательные поля, которые будут описаны в разделе моделей. Создатель задачи автоматически добавляется, как её участник.

После отправки формы на сервис задач данные проверяются на корректность и в случае возникновения ошибки генерируется исключение, формируется запись об ошибке в журнал сообщений и возвращается код ошибки пользователю с кратким описанием ошибки.

3.2 Модели данных

Основными моделями в программном средстве «Файловый менеджер с возможностью просмотра файлов на устройствах в одной локальной сети» являются:

- модель пользователя;
- модель задачи;

Для описания моделей использовалась библиотека Sequelize [11], которая написана для удобства работы с базой данных PostgreSQL.

3.2.1 Модель пользователя

Модель пользователя используется в сервисе аутентификации и содержит следующие поля:

- логин
- пароль
- имя
- фамилия
- почта, номер телефона, роль

Поля почты, номера телефона и роли используются, как вспомогательные. Используя почту или номер телефона пользователь быстро может связаться с коллегой. Роль может дать понять пользователям, чем занимается сотрудник.

3.2.2 Модель задачи

Модель задачи используется в сервисе задач и содержит в себе ассоциации на другие модели. Описание модели задачи:

- название и описание
- приоритет (ассоциация)
- статус (ассоциация)
- дата начала и дата окончания
- участники и прикрепленные файлы

Ассоциации необходимы для удобного и гибкого масштабирования моделей. Приоритеты и статусы связываются с моделью задачи и в случае, если пользователь захочет изменить базовый набор этих моделей, то они автоматически будут применены к модели задачи. Ассоциация

производится, по уникальному идентификатору, который есть и во вспомогательных моделях и в полях основной модели.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

При разработке программного средства на этапе перехода к использованию API Gateway возникла проблема с проксированием запросов к API. По изначально неизвестной причине клиент возвращал ошибку CORS Policy, которую можно увидеть на рисунке 4.1.

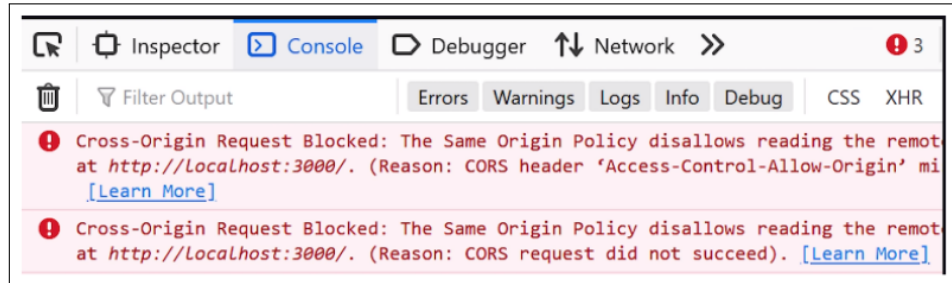


Рисунок 4.1 – Ошибка CORS Policy

После изучения проблемы и поиска её решения было выяснено, что проблема могла быть вызвана неправильной настройкой CORS в конкретном микросервисе к которому идёт запрос. Специально для этого была написана новая конфигурация CORS, которая была добавлена в проект. Код конфигурации можно увидеть ниже.

```
app.use(cors({
  origin: function(origin, callback) {
    // Разрешаем доступ для localhost и IP-адресов в диапазоне 192.168.*
    if (!origin ||
        /^http:\/\/localhost(:\d+)?$/.test(origin) ||
        /^http:\/\/192\.168\.(\d+)\.(\d+)(:\d+)?$/.test(origin)) {
      callback(null, true); // Разрешаем запросы
    } else {
      callback(new Error('Not allowed by CORS')); //
        Отказываем всем остальным
    }
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  credentials: true
}));
```

Данная конфигурация позволяет запросы только с localhost и IP-адресов в диапазоне 192.168.*. Это сделано для того, чтобы ограничить доступ к API только для локальной сети и localhost. В дальнейшем, при необходимости, можно будет добавить другие IP-адреса.

После внесения изменений в конфигурацию CORS проблема всё ещё не была решена. Тогда было принято решение изучить журнал сообщений API Gateway и микросервиса, к которому идёт запрос. В журнале API Gateway не было никакой записи о запросе, что указывало на то, что запрос не доходит до API Gateway и не обрабатывается микросервисом вовсе. Это может быть вызвано тем, что другое программное средство перекрывает запросы.

После изучения устройства было выяснено, что в системе установлена программа WSL [12] (Windows Subsystem for Linux), которая эмулирует Linux-систему на Windows. При этом, в этой виртуальной системе был установлен сервер Apache2, который по умолчанию запускался вместе с запуском системы. Проверить это удалось с помощью команды `sudo service apache2 status`, которая показала, что сервер Apache2 запущен и работает. Увидеть результат команды можно ниже.

```
kostya@WIN-A85PV5KAN9I:/mnt/c/Users/kostyabet$ sudo service apache2
status●
apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled;
         preset: enabled)
  Active: active (running) since Sat 2025-05-17 16:10:53 +03; 4s ago
  Docs: https://httpd.apache.org/docs/2.4/
  Process: 2204 ExecStart=/usr/sbin/apachectl start (code=exited,
         status=0/SUCCESS)
  Main PID: 2207 (apache2)
  Tasks: 9 (limit: 14999)
  Memory: 19.3M ()
  CGroup: /system.slice/apache2.service
          2207 /usr/sbin/apache2 -k start
          2209 /usr/sbin/apache2 -k start
          2210 /usr/sbin/apache2 -k start
          2211 /usr/sbin/apache2 -k start
          2212 /usr/sbin/apache2 -k start
          2213 /usr/sbin/apache2 -k start
          2216 /usr/sbin/apache2 -k start
          2217 /usr/sbin/apache2 -k start
          2218 /usr/sbin/apache2 -k start

May 17 16:10:53 WIN-A85PV5KAN9I systemd[1]: Starting apache2.service -
The Apache HTTP Server...
May 17 16:10:53 WIN-A85PV5KAN9I systemd[1]: Started apache2.service -
The Apache HTTP Server.
```

При этом, Apache2 использует 80 порт, который также используется API Gateway. Это и стало причиной возникновения проблемы, а так как Apache запускался автоматически, то он занимал порт 80 и не давал API Gateway его использовать. Таким образом, при попытке отправить запрос к API Gateway, запрос перенаправлялся на Apache2, который не знал, что с ним делать и возвращал ошибку CORS Policy. После остановки Apache2 проблема была решена и запросы начали обрабатываться API Gateway.

Для того чтобы в дальнейшем не возникало подобных проблем, было принято решение отключить автоматический запуск Apache2 при загрузке системы. Для этого была выполнена команда `sudo systemctl disable apache2`, которая отключает автоматический запуск Apache2. После этого, при перезагрузке системы, Apache2 не будет запускаться и 80 порт будет свободен.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Интерфейс программного средства

В интерфейсе программного средства имеются общие для всех страниц компоненты: панель настроек, навигационная панели. Помимо общих элементов, существуют и частные, которые описаны далее.

5.1.1 Страница регистрации

Страница регистрации – первая страница с которой сталкивается новый пользователь при входе в систему. Форма регистрации состоит из следующих компонентов:

- заголовок;
- форма регистрации;
- меню действий

Рассмотреть страницу регистрации можно на рисунке 5.1.

Рисунок 5.1 – Страница регистрации

Заголовок информирует пользователя о том, что за страница открыта и описывает действия, которые необходимо выполнить. Форма регистрации содержит в себе необходимые поля для создания нового пользователя и кнопку отправки формы регистрации. Меню действий необходимо, чтобы перейти на страницу входа, на случай, если пользователь попал на страницу регистрации случайно.

5.1.2 Профиль пользователя

Профиль пользователя – страница на которой можно узнать всю информацию о любом пользователе в системе. Владелец аккаунта может изменить свою контактную информацию на странице профиля пользователя.

Страница пользователя состоит из следующих частей:

- шапка страницы;

– меню;

Внешний вид страницы профиля пользователя можно увидеть на рисунке 5.2.

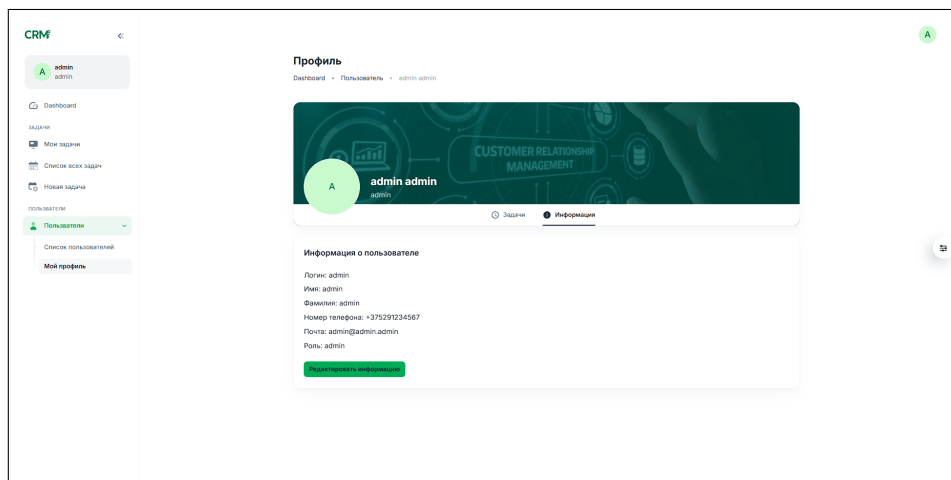


Рисунок 5.2 – Страница профиля пользователя

Шапка страницы показывает аватар, имя и роль пользователя. Навигационная панель позволяет пользователю выбрать, какую информацию необходимо вывести. Меню состоит из двух блоков: Задачи, Информация.

5.1.3 Страница создания задачи

Страница создания задачи является основной в программном средстве. Основные элементы из которых состоит страница:

- основные поля задачи;
- блок добавления пользователей;
- блок прикрепления файлов;

Рассмотреть страницу создания задачи можно на рисунке 5.3.

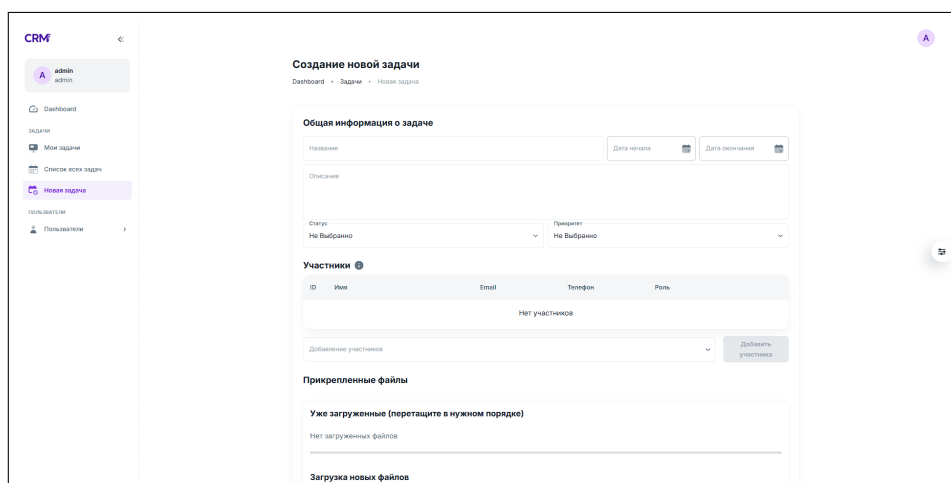


Рисунок 5.3 – Страница создания задачи

Основные поля задачи содержат всю информацию о задаче: название, сроки исполнения, приоритет и статус. Блок добавления задачи является таблицей пользователей, в которую можно добавлять и удалять участников задачи. Блок добавления файлов состоит из двух элементов: выбранные файлы и зона добавления файла с возможностью перетянуть в неё файл.

5.2 Управление программным средством

5.2.1 Вход в аккаунт

Перед началом работы с программным средством пользователю необходимо авторизоваться в системе. Необходимо ввести логин, пароль и нажать на кнопку отправки формы. Если данные валидны, то пользователя пропускают в систему, после чего можно ознакомиться со всем функционалом программного средства.

5.2.2 Создание новой задачи

Используя боковое меню в веб-приложении можно перемещаться между страницами программного средства. Выбрав там элемент «Новая задача» пользователь может создать новую задачу. У задачи можно указывать приоритет, статус, участников и файлы.

5.2.3 Просмотр всех задач

Для просмотра всех задач можно использовать основную страницу приложения, где в виде интерактивной панели где задачи группированы по приоритетам и статусам.

На странице задач пользователя можно просмотреть все задачи в которых участвует текущий пользователь. Информация отображается в виде таблицы со всей основной информацией. Нажав на строку с задачей, пользователь может изучить задачу подробнее. Помимо страницы задач пользователя существует страница всех задач на которой можно увидеть все задачи созданные в системе.

5.2.4 Настройки приложения

Особенностью программного средства является гибкая настройка внешнего вида. Пользователю доступны следующие настройки:

- смена цветовой палитры;
- изменение положения навигационной панели;
- смена вспомогательного цвета блоков;
- изменение основного шрифта;

Внешний вид панели настроек показан на рисунке 5.4.

Также пользователь может изменить положение всех блоков, контраст и компактность. Дополнительно, можно открыть веб-приложение в полноэкранном режиме.

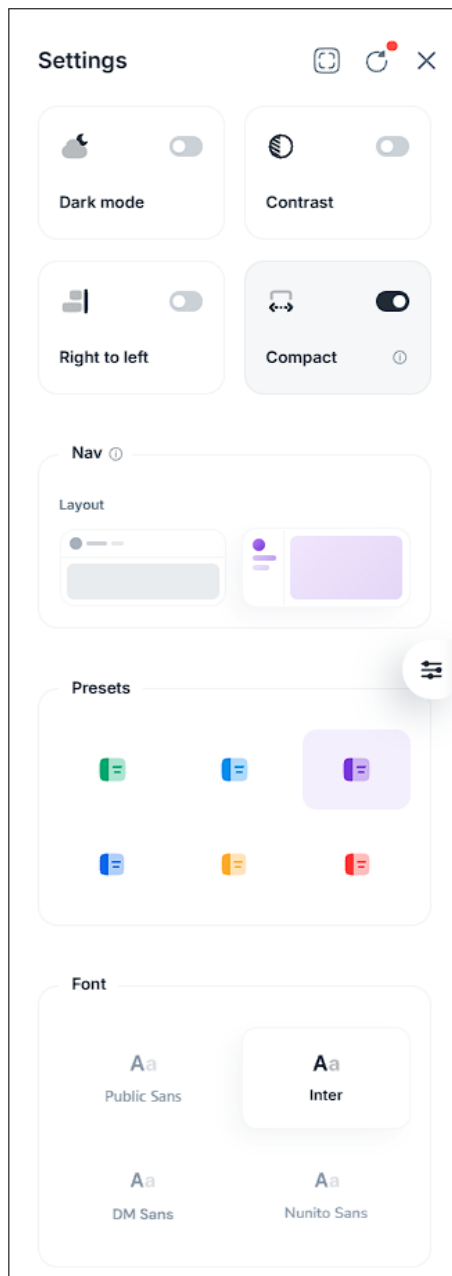


Рисунок 5.4 – Панель настроек программного средства

Все настройки сохраняются у пользователя на устройстве, что помогает не запоминать настройки, а изменить их единожды и изменять при необходимости. Последний введенные логин и пароль пользователем также сохраняется на устройстве для подстановки в момент следующего входа.

ЗАКЛЮЧЕНИЕ

В данном курсовом проекте было разработано программное средство «Файловый менеджер с возможностью просмотра файлов на устройствах в одной локальной сети», которое позволяет автоматизировать процесс управления клиентами и продажами в компании. В ходе работы над проектом были изучены основные аспекты разработки программного обеспечения, включая анализ требований, проектирование архитектуры, реализацию и тестирование.

Приложение создано с использованием современных технологий и инструментов, что позволяет обеспечить его высокую производительность и надежность. При разработке были реализованы следующие основные функции программного средства:

- сервис для работы с пользователями;
- сервис для работы с задачами;
- сервис веб-приложения;
- сервис базы данных;
- облачное файловое хранилище;
- сервис логирования;
- механизм межсервисного взаимодействия;
- алгоритм создания JWT токена;
- алгоритм сохранения файла в облачное хранилище;
- алгоритм отправки уведомления в систему логирования;

Для достижения поставленных целей были использованы современные подходы и методологии разработки программного обеспечения, такие как Agile, Scrum и DevOps. Это позволило обеспечить высокое качество программного продукта и его соответствие требованиям заказчика. В ходе работы над проектом были выявлены некоторые проблемы и ограничения, которые могут быть устранены в будущем. Например, добавление пагинации в сервис для работы с задачами, улучшение интерфейса веб-приложения и оптимизация работы с облачным хранилищем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Visual Studio Code documentation [Электронный ресурс]. — Режим доступа : <https://lasergbl.com>. — Дата доступа : 15.03.2025.
- [2] JavaScript documentation [Электронный ресурс]. — Режим доступа : <https://developer.mozilla.org/ru/docs/Web/JavaScript>. — Дата доступа : 25.02.2025.
- [3] Node.js documentation [Электронный ресурс]. — Режим доступа : <https://nodejs.org/docs/latest/api/>. — Дата доступа : 10.03.2025.
- [4] Docker Docs [Электронный ресурс]. — Режим доступа : <https://docs.docker.com/>. — Дата доступа : 03.04.2025.
- [5] nginx: документация [Электронный ресурс]. — Режим доступа : <https://nginx.org/ru/docs/>. — Дата доступа : 01.04.2025.
- [6] PostgreSQL documentation [Электронный ресурс]. — Режим доступа : <https://www.postgresql.org/docs/>. — Дата доступа : 28.03.2025.
- [7] Material-UI documentation [Электронный ресурс]. — Режим доступа : <https://mui.com/>. — Дата доступа : 18.02.2025.
- [8] Kafka documentation [Электронный ресурс]. — Режим доступа : <https://kafka.apache.org/documentation/>. — Дата доступа : 18.04.2025.
- [9] Kibana documentation [Электронный ресурс]. — Режим доступа : <https://www.elastic.co/docs/get-started/the-stack>. — Дата доступа : 01.05.2025.
- [10] JWT documentation [Электронный ресурс]. — Режим доступа : <https://jwt.io/introduction>. — Дата доступа : 12.04.2025.
- [11] Sequelize documentation [Электронный ресурс]. — Режим доступа : <https://sequelize.org/docs/v6/getting-started/>. — Дата доступа : 16.04.2025.
- [12] WSL documentation [Электронный ресурс]. — Режим доступа : <https://learn.microsoft.com/en-us/windows/wsl/>. — Дата доступа : 06.05.2025.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код

Services

```
export const setupInterceptors = (logout) => {
  httpClient.interceptors.response.use(
    (config) => config,
    async (error) => {
      const originalRequest = error.config;
      if (
        error.response?.status === 403 &&
        error.config &&
        !error.config._isRetry
      ) {
        try {
          if (isRefreshing) {
            return new Promise((resolve, reject) => {
              failedQueue.push({ reject, resolve });
            })
              .then((token) => {
                originalRequest.headers.Authorization = `Bearer ${token}`;
                return axios(originalRequest);
              })
              .catch((err) => Promise.reject(err));
          }
          originalRequest._isRetry = true;
          isRefreshing = true;
          const refreshTokenStorage = localStorage.getItem('refreshToken');
          return new Promise((resolve, reject) => {
            axios
              .post(`${API_URL}/auth/refresh`, {
                token: refreshTokenStorage,
              })
              .then(({ data }) => {
                window.localStorage.setItem('accessToken', data.accessToken);
                window.localStorage.setItem('refreshToken', data.refreshToken);
                httpClient.defaults.headers.common.Authorization = `Bearer ${data.accessToken}`;
                originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
                processQueue(null, data.accessToken);
                resolve(httpClient(originalRequest));
              })
              .catch((err) => {
                processQueue(err, null);
                logout();
                reject(err);
              })
              .finally(() => {
                isRefreshing = false;
              });
          });
        } catch (e) {
          console.error('Token expired', e);
        }
      }
    }
  );
}
```

```

        logout(); // fallback logout
      }
    }
    throw error;
  }
};

httpClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('accessToken');
  config.headers.Authorization = token ? `Bearer ${token}` : '';
  return config;
});

const generateToken = (user) => {
  try {
    const secret = { id: user.id, login: user.login };
    const accessToken = jwt.sign(
      secret,
      accessTokenSecret,
      { expiresIn: '15m' }
    );

    const refreshToken = jwt.sign(
      secret,
      refreshTokenSecret,
      { expiresIn: '30m' }
    );

    info('Сгенерированы новые токены .');
    return { accessToken, refreshToken };
  }
  catch (error) {
    throw new Error('Ошибка генерации токена : ' + error.message);
  }
};

exports.register = async (req, res) => {
  try {
    const { login, password, firstName, lastName, phone, email, role } =
      req.body;

    if (!login || !password || !firstName || !lastName || !phone || !
      email) {
      warn('Пожалуйста, заполните все обязательные поля .');
      return res.status(400).json({ message: 'Пожалуйста,
        заполните все обязательные поля .' });
    }

    const existing = await User.findOne({ where: { login } });

    if (existing) {
      warn('Пользователь с таким login: ${login} уже существует .');
      return res.status(400).json({ message: 'Пользователь с таким login
        : ${login} уже существует .' });
    }

    const avatarPath = req?.file
      ? req?.file?.path || null
      : null;
    const hash_password = await bcrypt.hash(password, 10);

```

```

const user = await User.create({
  login,
  password: hash_password,
  firstName,
  lastName,
  phone,
  email,
  role,
  photoURL: avatarPath,
});

info('Пользователь' успешнозарегистрирован !', { user: user.id });
res.status(201).json({
  message: 'Пользователь' создан',
  user: {
    id: user.id,
    login: user.login,
    firstName: user.firstName,
    lastName: user.lastName,
    email: user.email,
    phone: user.phone,
    role: user.role,
    avatar: user.photoURL,
  }
});
} catch (error) {
  console.error('Ошибка' регистрации:', error);
  errlog('Ошибка' регистрации: ${error.message}');
  res.status(500).json({ message: 'Ошибка' сервера' });
}
}

exports.login = async (req, res) => {
  try {
    const { login, pass: password } = req.body;

    if (!login || !password) {
      warn('Пожалуйста', заполнитевсеобязательныеполя .');
      return res.status(400).json({ message: 'Пожалуйста',
        заполнитевсеобязательныеполя .' });
    }

    const user = await User.findOne({ where: { login } });
    if (!user) {
      warn('Пользователь' с login: ${login} не найден .');
      return res.status(401).json({ message: 'Пользователь' с login: ${
        login} не найден .' });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      warn('Неверный' парольдляпользователяс login: ${login}');
      return res.status(401).json({ message: 'Неверный' пароль' });
    }

    const { accessToken, refreshToken } = generateToken(user);

    const userData = user.get({ plain: true });
    if (!userData) {
      return new Error({ message: 'Ошибка' полученияданных ' });
    }
  }
}

```

```

    }
    delete userData.password;

    info('Пользователь' успешно вошёл в систему !', { user: userData.id })
    ;
    res.status(200).json({
      message: 'Успешный вход',
      user: userData,
      accessToken,
      refreshToken,
    });
  } catch (error) {
    console.error('Ошибка входа:', error);
    errlog('Ошибка входа: ${error.message}');
    res.status(500).json({ message: 'Ошибка сервера' });
  }
}

exports.refreshToken = async (req, res) => {
  const { token: refreshToken } = req.body;

  if (!refreshToken) {
    warn('Refresh токен отсутствует ');
    return res.status(401).json({ message: 'Refresh токен отсутствует '
    });
  }

  try {
    const decoded = jwt.verify(refreshToken, refreshTokenSecret);

    const { accessToken: newAccessToken, refreshToken: newRefreshToken }
      = generateToken({
        id: decoded?.id,
        login: decoded?.login
      });

    info('Refresh токен успешно обновлён ');
    res.status(200).json({
      accessToken: newAccessToken,
      refreshToken: newRefreshToken
    });
  } catch (error) {
    console.error('Ошибка обновления токена:', error);
    errlog('Ошибка обновления токена . ${error}');
    return res.status(403).json({ message: 'Неверный или просроченный
      refresh токен' });
  }
}

async function log(level, message, extra = {}) {
  if (!isConnected) {
    await producer.connect();
    isConnected = true;
  }

  const logEntry = {
    timestamp: new Date().toISOString(),
    service: 'auth-service',
    level,
    message,
  }

```

```

    ...extra,
  };

  await producer.send({
    topic: 'logs',
    messages: [{ value: JSON.stringify(logEntry) }],
  });
}

function info(msg, extra) {
  return log('INFO', msg, extra);
}

function error(msg, extra) {
  return log('ERROR', msg, extra);
}

function warn(msg, extra) {
  return log('WARN', msg, extra);
}

function debug(msg, extra) {
  return log('DEBUG', msg, extra);
}

async function run() {
  await consumer.connect();
  await producer.connect();
  await consumer.subscribe({ topic: 'check-users' });

  await consumer.run({
    eachMessage: async ({ message }) => {
      const { users, correlationId } = JSON.parse(message.value.toString());

      const { exists } = await checkUsersExists(users);

      if (!exists) {
        warn('User(s) ${users} do not exist');
      }

      await producer.send({
        topic: 'check-users-response',
        messages: [
          {
            key: correlationId,
            value: JSON.stringify({ users, exists, correlationId })
          }
        ]
      });
    }
  });
}

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader?.split(' ')[1];

  if (!token) {
    warn('Токен отсутствует');
  }
}

```

```

    return res.status(401).json({ message: Токен' отсутствует' });
  }

  jwt.verify(token, accessTokenSecret, (err, decoded) => {
    if (err) {
      warn(Недействительный' токен');
      return res.status(403).json({ message: Недействительный' токен' });
    }

    debug(Токен' действителен');
    req.user = decoded;
    next();
  });
};

const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: {
    folder: 'avatars',
    allowed_formats: ['jpg', 'png', 'jpeg'],
  },
});

const upload = multer({ storage });

exports.create = async (req, res) => {
  try {
    const { name, description, startAt, endAt, priority, state } = req.body;
    const attachments = req.files?.map(file => file.path) || null;

    if (!name || !startAt || !endAt || !priority || !state) {
      warn(Не' всеобязательныеполязаполнены .');
      return res.status(400).json({ message: Не' всеобязательныеполязаполнены .' });
    }

    // Date check
    const start = new Date(startAt);
    const end = new Date(endAt);
    if (isNaN(start) || isNaN(end)) {
      warn(Неверный' форматдаты .');
      return res.status(400).json({ message: Неверный' форматдаты .' });
    }
    if (start >= end) {
      warn(Дата' началадолжнабытьраньшедатыокончания .');
      return res.status(400).json({ message: Дата' началадолжнабытьраньшедатыокончания .' });
    }

    // Users check
    const users = Array.isArray(req.body.users)
      ? req.body.users
      : [req.body.users];

    if (!users || users.length === 0) {
      warn(Пользователи' неуказаны .');
      return res.status(400).json({ message: Пользователи' неуказаны .' });
    }
  }
};

```

```

    }

    const correlationId = uuidv4();
    const responsePromise = createResponseWaiter(correlationId);
    await checkUsers(users, correlationId);

    const response = await responsePromise;
    const { exists } = response;

    const task = await Task.create({
      name,
      description,
      users: !exists || exists.length === 0 ? null : exists,
      attachments,
      startAt,
      endAt,
      priority,
      state
    });

    info('Задача' создана', { taskId: task.id })
    res.status(201).json({
      message: 'Задача' создана',
      task: task
    });
  } catch(error) {
    console.error('Ошибка' создания задания :', error);
    errlog('Ошибка' создания задания . ${error}');
    res.status(500).json({ message: 'Ошибка' сервера' });
  }
}

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <!-- Favicon -->
    <link
      rel="apple-touch-icon"
      sizes="180x180"
      href="/favicon/apple-touch-icon.png"
    />
    <link
      rel="icon"
      type="image/png"
      sizes="32x32"
      href="/favicon/favicon-32x32.png"
    />
    <link
      rel="icon"
      type="image/png"
      sizes="16x16"
      href="/favicon/favicon-16x16.png"
    />
    <meta name="theme-color" content="#000000" />
    <link rel="manifest" href="./manifest.json" />
    <!-- Using Google Font -->
    <link rel="preconnect" href="https://fonts.gstatic.com" />
    <link

```

```

        href="https://fonts.googleapis.com/css2?family=Public+Sans:
          wght@400;500;600;700&display=swap"
        rel="stylesheet"
      />
    <link
      href="https://fonts.googleapis.com/css2?family=Inter:wght@400
        ;500;600;700&display=swap"
      rel="stylesheet"
    />
    <link
      href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400
        ;500;600;700&display=swap"
      rel="stylesheet"
    />
    <link
      href="https://fonts.googleapis.com/css2?family=Nunito+Sans:
        wght@400;500;600;700&display=swap"
      rel="stylesheet"
    />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <title>CRM system</title>
  </head>
  <body>
    <div id="root"></div>
    <script src="/env-config.js"></script>
    <script type="module" src="/src/application/root.jsx"></script>
  </body>
</html>

```

```

export const fetchUserInfo = async () => {
  const response = await httpClient.get('/auth/me');
  return response.data;
};

export const fetchUpdateUser = async (userData) => {
  const response = await httpClient.put('/auth/me', userData);
  return response.data;
}

export const fetchUsersList = async () => {
  const response = await httpClient.get('/auth/getAllUsers');
  return response.data;
}

export const fetchUserInfoById = async (id) => {
  const response = await httpClient.get(`/auth/getUser/${id}`);
  return response.data;
}

```

Server settings

```

const app = express();
app.use(bodyParser.json());
app.use(cors({
  origin: function(origin, callback) {
    // Разрешаем доступ для localhost и IP-адресов в диапазоне 192.168.*
    if (!origin ||

```

```

        /^http:\/\/localhost(:\d+)?$/ .test(origin) ||
        /^http:\/\/192\.168\.\d+\.\d+(:\d+)?$/ .test(origin)) {
    callback(null, true); // Разрешаем запросы
} else {
    callback(new Error('Not allowed by CORS')); //
        Отказываем всем остальным
}
},
credentials: true
}});

```

Logstash.conf

```

input {
  kafka {
    bootstrap_servers => "kafka:9092"
    topics => ["logs"]
    group_id => "logstash"
  }
}

filter {
  json {
    source => "message"
  }

  mutate {
    add_field => {
      "message" => "%{[message]}"
      "service" => "%{[service]}"
      "level" => "%{[level]}"
    }
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    data_stream => "true"
    data_stream_type => "logs"
    data_stream_dataset => "all"
    data_stream_namespace => "prod"
  }
}

```

Docker-compose

```

services:
  logstash:
    depends_on:
      - elasticsearch
      - kafka
    container_name: logstash
    image: docker.elastic.co/logstash/logstash:8.5.3
    networks:
      - app-net
    environment:
      - "LOGSTASH_HTTP_HOST=0.0.0.0"
    ports:
      - "5044:5044"

```

```

volumes:
  - ./logstash/pipeline/logstash-prod.conf:/usr/share/logstash/
    pipeline/logstash.conf

elasticsearch:
  container_name: elasticsearch
  image: docker.elastic.co/elasticsearch/elasticsearch:8.5.3
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - xpack.security.http.ssl.enabled=false
    - ELASTIC_PASSWORD=admin
  networks:
    - app-net
  ports:
    - "9200:9200"
  volumes:
    - es_data:/usr/share/elasticsearch/data

kibana:
  container_name: kibana
  image: docker.elastic.co/kibana/kibana:8.5.3
  networks:
    - app-net
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  ports:
    - "5601:5601"
  volumes:
    - kibana_config:/usr/share/kibana/config

zookeeper:
  container_name: zookeeper
  image: confluentinc/cp-zookeeper:latest
  networks:
    - app-net
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

kafka:
  container_name: kafka
  depends_on:
    - zookeeper
  image: confluentinc/cp-kafka:latest
  ports:
    - "9092:9092"
  networks:
    - app-net
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  healthcheck:
    test: ["CMD", "kafka-topics", "--bootstrap-server", "localhost:9092", "--list"]
    interval: 10s
    timeout: 5s
    retries: 10

```

```

gateway:
  image: nginx:latest
  container_name: gateway
  ports:
    - "80:80"
  networks:
    - app-net
  volumes:
    - ./nginx/nginx.prod.conf:/etc/nginx/nginx.conf
  depends_on:
    - frontend
    - auth-service
    - tasks-service
    - log-service

db:
  image: postgres:14
  container_name: postgres-db
  restart: always
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: postgres
  ports:
    - "5432:5432"
  networks:
    - app-net
  volumes:
    - postgres-data:/var/lib/postgresql/data

log-service:
  image: kostyabet228/crm-system:log_service-latest
  container_name: log-service
  networks:
    - app-net
  expose:
    - "5002"
  depends_on:
    db:
      condition: service_started
    kafka:
      condition: service_healthy
  restart: on-failure
  environment:
    ACCESSTOKENSECRET: ${ACCESSTOKENSECRET}
    REFRESHTOKENSECRET: ${REFRESHTOKENSECRET}
    KAFKA_BROKER: kafka:9092

auth-service:
  image: kostyabet228/crm-system:auth_service-latest
  container_name: auth-service
  networks:
    - app-net
  expose:
    - "5000"
  depends_on:
    db:
      condition: service_started
    kafka:
      condition: service_healthy

```

```

restart: on-failure
environment:
  ACCESSTOKENSECRET: ${ACCESSTOKENSECRET}
  REFRESHTOKENSECRET: ${REFRESHTOKENSECRET}
  DB_USER: ${DB_USER}
  DB_PASSWORD: ${DB_PASSWORD}
  DB_HOST: ${DB_HOST}
  DB_PORT: ${DB_PORT}
  DB_NAME: ${DB_NAME}
  CLOUDINARY_CLOUD_NAME: ${CLOUDINARY_CLOUD_NAME}
  CLOUDINARY_API_KEY: ${CLOUDINARY_API_KEY}
  CLOUDINARY_API_SECRET: ${CLOUDINARY_API_SECRET}
  KAFKA_BROKER: kafka:9092

tasks-service:
  image: kostyabet228/crm-system:tasks_service-latest
  container_name: tasks-service
  networks:
    - app-net
  expose:
    - "5001"
  depends_on:
    db:
      condition: service_started
    kafka:
      condition: service_healthy
  restart: on-failure
  environment:
    ACCESSTOKENSECRET: ${ACCESSTOKENSECRET}
    REFRESHTOKENSECRET: ${REFRESHTOKENSECRET}
    DB_USER: ${DB_USER}
    DB_PASSWORD: ${DB_PASSWORD}
    DB_HOST: ${DB_HOST}
    DB_PORT: ${DB_PORT}
    DB_NAME: ${DB_NAME}
    GATEWAY_URL: http://gateway:80
    KAFKA_BROKER: kafka:9092

frontend:
  image: kostyabet228/crm-system:frontend-latest
  container_name: frontend
  networks:
    - app-net
  ports:
    - "3000:3000"
  environment:
    VITE_API_LOCAL: http://gateway:80

networks:
  app-net:
    driver: bridge

volumes:
  kibana_config:
  es_data:
  postgres-data: {}

```

Nginx.conf

```

events {}

```

```

http {
    upstream auth {
        server auth-service:5000;
    }

    upstream tasks {
        server tasks-service:5001;
    }

    upstream log {
        server log-service:5002;
    }

    server {
        listen 80;

        location ~ ^/(tasks|tasks/|priority|priority/|state|state/) {
            proxy_pass http://tasks;
            proxy_redirect off;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location ~ ^/(auth|uploads/avatars)/ {
            proxy_pass http://auth;
            proxy_redirect off;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location ~ ^/(log|log/) {
            proxy_pass http://log;
            proxy_redirect off;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}

```